

MODUL 6

Nama: Ryan Akeyla Novianto Widodo

Kelas: 12 IF 07

NIM : 103112400081

DASAR TEORI

Dasar Teori C++

Diambil dari internet dan dirangkum ulang oleh saya dan dibantu dengan AI.

1. Inti Program

- Syntax: Mirip bahasa Inggris, tapi huruf besar dan kecil beda arti. Setiap perintah biasanya diakhiri dengan titik koma (;).

- Komentar: Catatan di kode yang tidak dijalankan.

- Satu baris: // Ini komentar

- Banyak baris: /* Ini komentar yang panjang */

- Header: File yang berisi daftar perintah dan fungsi yang bisa dipakai. Contoh:

cpp

```
#include <iostream> // Untuk perintah input/output (cout, cin)
```

- main() : Fungsi utama, tempat program mulai berjalan.

cpp

```
int main() {
```

```
    // Kode program di sini
```

```
    return 0; // Menandakan program selesai dengan benar
```

```
}
```

- Namespace: Seperti "area" untuk nama-nama perintah. std adalah namespace standar C++.

cpp

```
using namespace std; // Supaya tidak perlu tulis std::cout, std::cin, dll.
```

- Input/Output:

- cout : Menampilkan sesuatu di layar.

- cin : Membaca input dari keyboard.

cpp

```
cout << "Halo!" << endl; // Menampilkan "Halo!"
```

```
int umur;
```

```
cin >> umur; // Membaca angka dari keyboard dan disimpan di variabel "umur"
```

2. Jenis Data

- Integer (Bilangan Bulat):

- int : Bilangan bulat (contoh: -10, 0, 25).

- short : Bilangan bulat pendek.

- long : Bilangan bulat panjang.

- Floating-Point (Bilangan Desimal):

- float : Bilangan desimal (contoh: 3.14).

- double : Bilangan desimal dengan ketelitian lebih tinggi.

- Character (Karakter):

- char : Satu huruf, angka, atau simbol (contoh: 'a', 'Z', '5').

- Boolean (Logika):

- bool : Nilai benar (true) atau salah (false).

- void : "Kosong". Dipakai untuk fungsi yang tidak menghasilkan nilai.

3. Variabel

- Deklarasi: Memberi tahu program nama dan jenis data variabel.

cpp

```
int umur; // Variabel bernama "umur" untuk menyimpan bilangan bulat
```

```
double harga; // Variabel bernama "harga" untuk menyimpan bilangan desimal
```

- Inisialisasi: Memberi nilai awal ke variabel.

cpp

```
int umur = 20; // "umur" diisi dengan nilai 20
```

```
double harga = 99.50; // "harga" diisi dengan nilai 99.50
```

4. Operator

- Aritmatika: + (tambah), - (kurang), * (kali), / (bagi), % (sisanya).
- Penugasan: = (mengisi nilai ke variabel).
- Perbandingan: == (sama dengan), != (tidak sama dengan), > (lebih besar), < (lebih kecil), >= (lebih besar atau sama dengan), <= (lebih kecil atau sama dengan).
- Logika: && (DAN), || (ATAU), ! (TIDAK).
- Increment/Decrement: ++ (tambah 1), -- (kurang 1).

5. Struktur Kontrol

- if : Melakukan sesuatu jika kondisi benar.

cpp

```
if (umur >= 17) {  
    cout << "Anda sudah dewasa." << endl;  
} else {  
    cout << "Anda belum dewasa." << endl;  
}
```

- switch : Memilih salah satu dari beberapa pilihan.

cpp

```
int hari = 3;  
switch (hari) {  
    case 1:  
        cout << "Senin" << endl;  
        break;  
    case 2:
```

```

        cout << "Selasa" << endl;

        break;

    case 3:

        cout << "Rabu" << endl;

        break;

    default:

        cout << "Hari lain" << endl;

}

```

- for : Melakukan sesuatu berulang kali sebanyak yang ditentukan.

cpp

```

for (int i = 0; i < 5; i++) {

    cout << i << " "; // Menampilkan angka 0 sampai 4

}

cout << endl;

```

- while : Melakukan sesuatu berulang kali selama kondisi benar.

cpp

```

int angka = 1;

while (angka <= 3) {

    cout << angka << " ";

    angka++;

}

cout << endl;

```

- do-while : Sama seperti while , tapi dilakukan minimal sekali.

cpp

```

int pilihan;

do {

    cout << "Pilih 1, 2, atau 3: ";

```

```
cin >> pilihan;  
} while (pilihan < 1 || pilihan > 3);
```

6. Fungsi

- Definisi: Sekumpulan kode yang melakukan tugas tertentu.

cpp

```
int tambah(int a, int b) {  
    return a + b;  
}
```

- Pemanggilan: Menggunakan fungsi.

cpp

```
int hasil = tambah(5, 2); // Memanggil fungsi "tambah" dengan angka 5 dan 2  
cout << "Hasil: " << hasil << endl; // Menampilkan "Hasil: 7"
```

7. Pointer

- Definisi: Variabel yang menyimpan alamat memori variabel lain. (Agak rumit, tapi penting).

cpp

```
int umur = 25;  
  
int *p_umur = &umur; // p_umur menyimpan alamat memori dari variabel umur
```

DASAR TEORI DOUBLY LINKED LIST

Pengertian Doubly Linked List

Doubly Linked List (DLL) adalah struktur data linear yang terdiri dari sekumpulan elemen yang disebut node. Setiap node dalam DLL berisi:

- Data (Info): Informasi yang disimpan dalam node.
- Pointer ke Node Berikutnya (Next): Alamat memori dari node selanjutnya dalam urutan.
- Pointer ke Node Sebelumnya (Prev): Alamat memori dari node sebelumnya dalam urutan.

Dengan adanya pointer `next` dan `prev`, DLL memungkinkan traversal (penelusuran) list dalam dua arah: maju (dari head ke tail) dan mundur (dari tail ke head).

Perbedaan dengan Singly Linked List

Perbedaan utama antara DLL dan Singly Linked List (SLL) adalah keberadaan pointer `prev` . SLL hanya memiliki pointer `next` , sehingga hanya memungkinkan traversal satu arah.

Karakteristik Doubly Linked List

- Dinamis: Ukuran list dapat bertambah atau berkurang saat runtime.
- Non-Contiguous: Elemen-elemen list tidak harus disimpan dalam lokasi memori yang berurutan.
- Traversal Dua Arah: Memungkinkan penelusuran list dari depan ke belakang atau sebaliknya.
- Membutuhkan Lebih Banyak Memori: Setiap node membutuhkan memori tambahan untuk menyimpan pointer `prev` .

Operasi pada Doubly Linked List

Berikut adalah beberapa operasi umum yang dapat dilakukan pada DLL:

- Insert (Penyisipan):
 - `insertFirst(data)` : Menyisipkan node baru di awal list.
 - `insertLast(data)` : Menyisipkan node baru di akhir list.
 - `insertAfter(node, data)` : Menyisipkan node baru setelah node tertentu.
 - `insertBefore(node, data)` : Menyisipkan node baru sebelum node tertentu.
- Delete (Penghapusan):
 - `deleteFirst()` : Menghapus node pertama dari list.
 - `deleteLast()` : Menghapus node terakhir dari list.
 - `deleteNode(node)` : Menghapus node tertentu dari list.
- Search (Pencarian): Mencari node dengan data tertentu dalam list.
- Traversal (Penelusuran): Mengunjungi setiap node dalam list.
- `IsEmpty`: Memeriksa apakah list kosong.
- `GetSize`: Mendapatkan jumlah node dalam list.

Keuntungan Doubly Linked List

- Traversal Dua Arah: Memudahkan operasi yang membutuhkan penelusuran mundur.

- Penghapusan Lebih Efisien: Penghapusan node lebih efisien karena kita memiliki akses langsung ke node sebelumnya.
- Implementasi Operasi Tertentu Lebih Mudah: Beberapa operasi, seperti `insertBefore`, lebih mudah diimplementasikan.

Kerugian Doubly Linked List

- Membutuhkan Lebih Banyak Memori: Setiap node membutuhkan memori tambahan untuk pointer `prev`.
- Operasi Lebih Kompleks: Operasi penyisipan dan penghapusan lebih kompleks dibandingkan SLL karena kita perlu memperbarui pointer `prev` dan `next`.

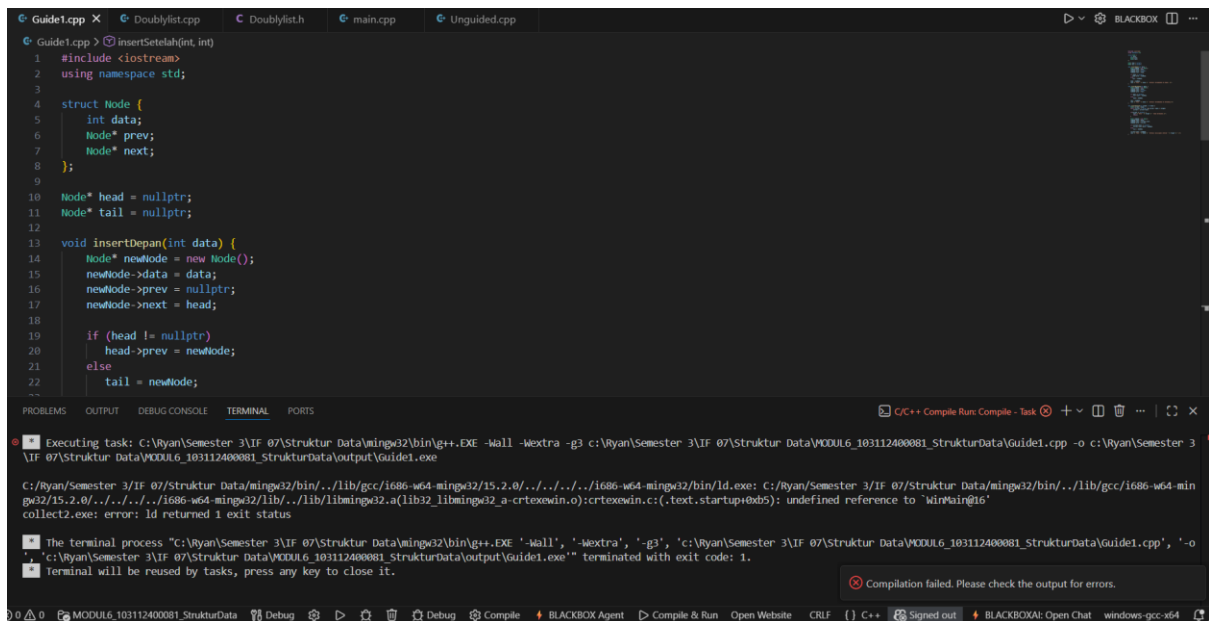
Implementasi Doubly Linked List

DLL dapat diimplementasikan dalam berbagai bahasa pemrograman, seperti C++, Java, Python, dll. Implementasi biasanya melibatkan definisi struktur data untuk node dan kelas untuk list itu sendiri, yang berisi fungsi-fungsi untuk melakukan operasi pada list. Contoh implementasi C++ sudah saya berikan sebelumnya.

Contoh Penggunaan Doubly Linked List

- Implementasi Undo/Redo: DLL dapat digunakan untuk menyimpan riwayat tindakan dalam aplikasi, sehingga memungkinkan pengguna untuk membatalkan (undo) atau mengulangi (redo) tindakan mereka.
- Navigasi Web: Browser web dapat menggunakan DLL untuk menyimpan riwayat halaman yang dikunjungi, sehingga memungkinkan pengguna untuk maju dan mundur dengan mudah.
- Music Playlist: Aplikasi pemutar musik dapat menggunakan DLL untuk mengelola daftar putar musik, sehingga memungkinkan pengguna untuk berpindah ke lagu sebelumnya atau berikutnya dengan mudah.

Guided 1



The screenshot shows a C++ IDE with a file named `Guide1.cpp`. The code defines a `Node` struct with `data`, `prev`, and `next` pointers. It initializes `head` and `tail` to `nullptr`. The `insertDepan` function inserts a new node at the beginning of the list. The terminal shows a compilation error: `undefined reference to 'winMain@16'`, which is a common issue when using MinGW without a proper main function or linker options.

```
#include <iostream>
#include <string>
using namespace std;

struct Node {
    int data;
    Node* prev;
    Node* next;
};

Node* head = nullptr;
Node* tail = nullptr;

void insertDepan(int data) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->prev = nullptr;
    newNode->next = head;

    if (head != nullptr)
        head->prev = newNode;
    else
        tail = newNode;
}
```

Compiling failed. Please check the output for errors.

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita melakukan manipulasi dasar pada sebuah single linked list

Unguided

Coding:

```
#include <iostream>
#include <string>
using namespace std;

struct Kendaraan {
    string nomorPolisi;
    string warna;
    int tahun;
};

typedef Kendaraan InfoKendaraan;

struct Node {
    InfoKendaraan data;
    Node* next;
    Node* prev;
};

typedef Node* Address;

struct List {
```



```

    Address first;
    Address last;
};

void buatListKosong(List &daftarKendaraan);
Address buatNodeBaru(InfoKendaraan kendaraanBaru);
void hapusNode(Address node);
void tambahKendaraanDiAkhir(List &daftarKendaraan, Address nodeBaru);
void tampilkanKendaraan(const List &daftarKendaraan);
Address cariKendaraan(const List &daftarKendaraan, const string &nomorPolisi);
void hapusKendaraanPertama(List &daftarKendaraan, Address &node);
void hapusKendaraanTerakhir(List &daftarKendaraan, Address &node);
void hapusKendaraanSetelah(Address sebelum, Address &node);
bool cekDuplikat(const List &daftarKendaraan, const string &nomorPolisi);

void buatListKosong(List &daftarKendaraan) {
    daftarKendaraan.first = nullptr;
    daftarKendaraan.last = nullptr;
}

Address buatNodeBaru(InfoKendaraan kendaraanBaru) {
    Address node = new Node;
    node->data = kendaraanBaru;
    node->next = nullptr;
    node->prev = nullptr;
    return node;
}

void hapusNode(Address node) {
    if (node != nullptr) {
        delete node;
    }
}

void tambahKendaraanDiAkhir(List &daftarKendaraan, Address nodeBaru) {
    if (daftarKendaraan.first == nullptr) {
        daftarKendaraan.first = nodeBaru;
        daftarKendaraan.last = nodeBaru;
    } else {
        daftarKendaraan.last->next = nodeBaru;
        nodeBaru->prev = daftarKendaraan.last;
        daftarKendaraan.last = nodeBaru;
    }
}

void tampilkanKendaraan(const List &daftarKendaraan) {
    Address node = daftarKendaraan.first;
    if (node == nullptr) {
        cout << "\nDATA LIST KOSONG\n";
        return;
    }
}

```

```

    }
    cout << "\nDATA LIST\n";
    while (node != nullptr) {
        cout << "Nomor Polisi : " << node->data.nomorPolisi << endl;
        cout << "Warna      : " << node->data.warna << endl;
        cout << "Tahun      : " << node->data.tahun << endl << endl;
        node = node->next;
    }
}

Address cariKendaraan(const List &daftarKendaraan, const string &nomorPolisi) {
    Address node = daftarKendaraan.first;
    while (node != nullptr) {
        if (node->data.nomorPolisi == nomorPolisi) {
            return node;
        }
        node = node->next;
    }
    return nullptr;
}

void hapusKendaraanPertama(List &daftarKendaraan, Address &node) {
    node = nullptr;
    if (daftarKendaraan.first == nullptr) return;

    node = daftarKendaraan.first;
    if (daftarKendaraan.first == daftarKendaraan.last) {
        daftarKendaraan.first = nullptr;
        daftarKendaraan.last = nullptr;
    } else {
        daftarKendaraan.first = daftarKendaraan.first->next;
        daftarKendaraan.first->prev = nullptr;
    }
    node->next = nullptr;
    node->prev = nullptr;
}

void hapusKendaraanTerakhir(List &daftarKendaraan, Address &node) {
    node = nullptr;
    if (daftarKendaraan.last == nullptr) return;

    node = daftarKendaraan.last;
    if (daftarKendaraan.first == daftarKendaraan.last) {
        daftarKendaraan.first = nullptr;
        daftarKendaraan.last = nullptr;
    } else {
        daftarKendaraan.last = daftarKendaraan.last->prev;
        daftarKendaraan.last->next = nullptr;
    }
    node->next = nullptr;
}

```

```

    node->prev = nullptr;
}

void hapusKendaraanSetelah(Address sebelum, Address &node) {
    node = nullptr;
    if (sebelum == nullptr) return;
    if (sebelum->next == nullptr) return;

    node = sebelum->next;
    Address setelah = node->next;
    sebelum->next = setelah;
    if (setelah != nullptr) {
        setelah->prev = sebelum;
    }
    node->next = nullptr;
    node->prev = nullptr;
}

bool cekDuplikat(const List &daftarKendaraan, const string &nomorPolisi) {
    Address node = daftarKendaraan.first;
    while (node != nullptr) {
        if (node->data.nomorPolisi == nomorPolisi) {
            return true;
        }
        node = node->next;
    }
    return false;
}

int main() {
    List daftarKendaraan;
    buatListKosong(daftarKendaraan);

    int jumlah;
    cout << "Masukkan jumlah data kendaraan: ";
    if (!(cin >> jumlah) || jumlah < 0) {
        cout << "Input jumlah tidak valid.\n";
        return 0;
    }
    cout << endl;

    for (int i = 0; i < jumlah; i++) {
        InfoKendaraan kendaraanBaru;

        cout << "Masukkan nomor polisi: ";
        cin >> kendaraanBaru.nomorPolisi;

        if (cekDuplikat(daftarKendaraan, kendaraanBaru.nomorPolisi)) {
            cout << "Nomor polisi sudah terdaftar\n\n";
            i--;
        }
    }
}

```

```

        continue;
    }

    cout << "Masukkan warna kendaraan: ";
    cin >> kendaraanBaru.warna;
    cout << "Masukkan tahun kendaraan: ";
    cin >> kendaraanBaru.tahun;
    cout << endl;

    Address nodeBaru = buatNodeBaru(kendaraanBaru);
    tambahKendaraanDiAkhir(daftarKendaraan, nodeBaru);
}

tampilkanKendaraan(daftarKendaraan);

string nomorDicari;
cout << "Masukkan Nomor Polisi yang dicari : ";
cin >> nomorDicari;

Address ditemukan = cariKendaraan(daftarKendaraan, nomorDicari);
if (ditemukan != nullptr) {
    cout << "\nNomor Polisi : " << ditemukan->data.nomorPolisi << endl;
    cout << "Warna      : " << ditemukan->data.warna << endl;
    cout << "Tahun      : " << ditemukan->data.tahun << endl;
} else {
    cout << "Data tidak ditemukan.\n";
}

string nomorDihapus;
cout << "\nMasukkan Nomor Polisi yang akan dihapus : ";
cin >> nomorDihapus;

Address nodeDihapus = cariKendaraan(daftarKendaraan, nomorDihapus);
if (nodeDihapus != nullptr) {
    if (nodeDihapus == daftarKendaraan.first) {
        hapusKendaraanPertama(daftarKendaraan, nodeDihapus);
    } else if (nodeDihapus == daftarKendaraan.last) {
        hapusKendaraanTerakhir(daftarKendaraan, nodeDihapus);
    } else {
        Address prevNode = nodeDihapus->prev;
        hapusKendaraanSetelah(prevNode, nodeDihapus);
    }
    hapusNode(nodeDihapus);
    cout << "Data dengan nomor polisi " << nomorDihapus << " berhasil
dihapus.\n";
} else {
    cout << "Data tidak ditemukan.\n";
}

tampilkanKendaraan(daftarKendaraan);

```

```

Address cur = daftarKendaraan.first;
while (cur != nullptr) {
    Address next = cur->next;
    hapusNode(cur);
    cur = next;
}
daftarKendaraan.first = daftarKendaraan.last = nullptr;

return 0;
}

```

Hasil Coding

```

C:\Ryan\Semester 3\IF 07\Struktur Data\MODUL6_103112400081_StrukturData>cd "c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL6_103112400081_StrukturData\output"
c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL6_103112400081_StrukturData\output>.\"Unguided.exe"
Masukkan jumlah data kendaraan: 3

Masukkan nomor polisi: D001
Masukkan warna kendaraan: hitam
Masukkan tahun kendaraan: 90

Masukkan nomor polisi: D003
Masukkan warna kendaraan: putih
Masukkan tahun kendaraan: 70

Masukkan nomor polisi: D001
Nomor polisi sudah terdaftar

Masukkan nomor polisi: D004
Masukkan warna kendaraan: kuning
Masukkan tahun kendaraan: 90

DATA LIST
Nomor Polisi : D001
Warna       : hitam
Tahun       : 90

Nomor Polisi : D003
Warna       : putih
Tahun       : 70

Nomor Polisi : D004
Warna       : kuning
Tahun       : 90

Masukkan Nomor Polisi yang dicari : D001

Nomor Polisi : D001
Warna       : hitam
Tahun       : 90

Masukkan Nomor Polisi yang akan dihapus : D003
Data dengan nomor polisi D003 berhasil dihapus.

DATA LIST
Nomor Polisi : D001
Warna       : hitam
Tahun       : 90

Nomor Polisi : D004
Warna       : kuning
Tahun       : 90

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL6_103112400081_StrukturData\output>

```

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita melakukan pengelolaan data kendaraan menggunakan struktur data *Doubly Linked List*.